

Hooks, CI/CD e defesa em camadas contra código quebrado

Pesquisa sobre hooks, exit codes, branch protection rules e cultura de testes, motivada pela prática "O Especialista em Automação e o Líder de QA 'Escaldado'": como configurar o Claude Code para nunca marcar uma tarefa como concluída se testes, linter ou scan de segurança falharem — e como tornar esse bloqueio impossível de burlar.

Fernando Melo · 8 de setembro de 2026

01 · O PROBLEMA

Uma IA que compila código sem ser fiscalizada

O ponto de partida é a desconfiança de um líder de QA que já teve problemas com código gerado por IA entrando no repositório sem passar pela suíte de testes. A pergunta central: o que são os "Hooks" e como usá-los para forçar a IA a rodar a suíte de testes inteira antes de declarar uma tarefa concluída — e é possível travar a ferramenta se os testes falharem?

02 · HOOKS: O PORTEIRO DO EXIT CODE

Um gatilho que só abre a porta se o teste passar

Hooks são gatilhos automatizados disparados quando um evento específico ocorre (antes de um commit, ao receber um webhook). No contexto de IA, um hook de pré-execução ou pós-processamento impede a IA de dar a tarefa como concluída até o comando de teste retornar sucesso.

A mecânica depende do exit code do terminal: se o comando de teste falhar (código diferente de 0), o hook interrompe o fluxo — como um porteiro que só deixa passar quando o "código de saída" é positivo.

ABORDAGEM	ONDE SE APLICA	COMO FUNCIONA
Regras de projeto (IDE)	Cursor, Windsurf, Copilot Workspace	Arquivo .cursorrules/.windsurfrules instrui o agente a só declarar a tarefa concluída se os testes passarem.
Git Hook (pre-commit)	Qualquer repositório Git	Script em .git/hooks/pre-commit roda a suíte antes do commit; exit code diferente de 0 cancela o commit.
Flag nativa da CLI	Aider e ferramentas similares	--test-command roda o teste após cada alteração; se falhar, o erro volta para a IA corrigir sozinha.

03 · O LIMITE DO HOOK LOCAL

Localmente, nenhum bloqueio é 100% inviolável

Qualquer desenvolvedor com acesso ao terminal consegue burlar um hook local — com `git commit --no-verify` (que ignora os git hooks) ou simplesmente desativando as extensões da IDE. Para tornar o bloqueio impossível de ignorar, a validação precisa sair da máquina do desenvolvedor e migrar para o servidor onde o código é guardado.

CI/CD + Branch Protection Rules

O servidor atua como juiz final; os hooks locais são apenas o filtro inicial. A blindagem exige duas peças:

- 1 Configure uma pipeline de CI/CD (GitHub Actions, GitLab CI) que clona o projeto num ambiente isolado e roda a suíte completa a cada push ou Pull Request.
- 2 Require a pull request before merging — ninguém, nem a IA, pode dar git push direto na main; tudo passa por PR.
- 3 Require status checks to pass before merging — vincula o job do CI (ex.: run-tests) como trava do botão de merge.
- 4 Do not allow bypassing the above settings (Enforce administrations) — bloqueia até os administradores do repositório.

Trecho ilustrativo simplificado — o exemplo original de workflow foi cortado na borda da captura; a reconstrução abaixo preserva a ideia, não os caracteres exatos.

```
# .github/workflows/tests.yml
name: Testes Automatizados
on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Instalar dependências
        run: npm ci
      - name: Rodar testes
        run: npm test # build falha se o comando retornar erro
```

Nem "--no-verify" escapa do servidor

- 1 A IA (ou o dev) altera o código localmente.
- 2 Mesmo pulando o porteiro local com --no-verify, o commit ainda precisa virar um Pull Request para chegar na branch principal.
- 3 O CI entra em ação e roda a suíte de testes na nuvem.
- 4 Se falhar, o botão de merge fica travado — não importa o comando digitado no terminal do desenvolvedor.

O servidor central não aceita código quebrado, independentemente do que aconteceu na máquina local.

Auditar sem depender de confiança

Marca d'água no commit

O hook local de commit-msg/pre-commit injeta um metadado oculto quando roda com sucesso. Se o commit chegar ao PR sem essa marca, é sinal de --no-verify.

Husky "invisível"

Em vez de depender do dev lembrar de instalar os hooks, o script "prepare": "husky install" no package.json os ativa automaticamente a cada npm install.

Cruzamento Git vs. CI

Se o CI falha logo de cara em testes básicos que deveriam ter passado localmente, é sinal de hook pulado — algumas equipes automatizam um alerta pedindo para não usar --no-verify.

Fazer o hook ser mais rápido do que a tentação de pular

- 1 Taxa do CI (gamificação) — exibir o tempo de fila do CI nas dailies e notificar o time no Slack/Discord quando alguém quebra o pipeline por erro bobo.
- 2 Hooks "staged-only" e rápidos — lint-staged (JS) ou pre-commit (Python) rodam só nos arquivos alterados, caindo de minutos para segundos.
- 3 Separar lint/formatação de testes de verdade — o hook corrige automaticamente (eslint --fix, black) em vez de só travar e brigar por um espaço em branco.
- 4 IA como aliada no terminal — quando a própria ferramenta de IA se recusa a avançar se o teste quebrar, o dev se acostuma ao ciclo codificar → testar → corrigir antes do push.

Fecha com um acordo de time (Definition of Done):

"Nosso CI está ficando lento porque perdemos tempo esperando builds que falham por bobeira. O 'Pronto' de uma tarefa passa a significar que ela já passou nos testes locais — o CI serve para validar a integração final, não para ser o ambiente de testes principal."

Linter e scan de segurança antes de encerrar a tarefa

Como o Claude não roda sozinho na máquina, a trava é instruída via arquivo de regras do projeto ou parâmetros da CLI.

Trecho ilustrativo simplificado — as instruções de .cursorrules foram cortadas na borda da captura original; a versão abaixo recria a intenção, não o texto literal.

```
# Regras de Definição de Pronto
Sempre que terminar um bugfix ou refatoração:
1. Execute o linter do projeto.
2. Execute o scan de segurança/vulnerabilidades.

Comandos: npm run lint · npm audit

# Se qualquer comando retornar erro, corrija e rode de novo
# Só declare a tarefa concluída com os logs limpos
```

No Aider, o mesmo controle vira uma flag nativa, encadeando linter e scan com &&:

```
aider --test-command "npm run lint && npm audit" --auto-test
```

ECOSSISTEMA	LINTER	SCAN DE SEGURANÇA
JavaScript / TypeScript	eslint	npm audit ou snyk test
Python	flake8 ou ruff check	bandit -r . ou pip-audit
Go	golangci-lint run	govulncheck ./...
Docker / Infra (IaC)	hadolint Dockerfile	trivy config . ou checkov

CODEOWNERS: nem o próprio Claude pode se auto-liberar

Para impedir que alguém (ou a própria IA, induzida) altere as regras locais e remova as travas, o arquivo de configuração vira um item protegido no Git, com aprovação manual obrigatória:

```
# Qualquer alteração nas regras da IA precisa da sua aprovação
.cursorrules @seu_usuario_github
.aider.conf.yml @seu_usuario_github
```

Com o recurso de CODEOWNERS do GitHub/GitLab, qualquer PR que toque nesses arquivos exige assinatura manual de um arquiteto ou líder técnico antes do merge.

Defense in Depth

A validação local dá velocidade e feedback instantâneo; o CI/CD no servidor funciona como a instância final. Se algo passar quebrado localmente — por malícia, erro de configuração ou pressa — o servidor pega, barra o código e protege a branch principal.

Camada 1 — Local (ágil, mas burlável)

Claude, hooks e linter na máquina do dev. Rápida e dá feedback imediato, mas pode ser ignorada com `--no-verify` ou alteração do arquivo de regras.

Camada 2 — Servidor (lenta, porém inviolável)

CI/CD isolado + Branch Protection Rules. Ignora tudo que aconteceu na máquina do dev, roda testes, linter e scan do zero e bloqueia o merge se algo falhar.

Mesmo que o desenvolvedor seja o dono do projeto, com "Do not allow bypassing" ativo o GitHub/GitLab rejeita o código até que todos os testes fiquem verdes no servidor.